

CAGE: A Tool for Code Assessment and Grading

Ümit Kanoğlu

Computer Engineering, Ozyegin University

Istanbul, Türkiye
umit.kanoglu@ozu.edu.tr

Hasan Sözer

Computer Engineering, Ozyegin University

Istanbul, Türkiye
hasan.sozer@ozyegin.edu.tr

Oğuz Kerem Yıldız

Computer Engineering, Aşık University

Istanbul, Türkiye
oguzkeremyildiz@gmail.com

Olca Taner Yıldız

Computer Engineering, Ozyegin University

Istanbul, Türkiye
olcay.yildiz@ozyegin.edu.tr

Abstract—We introduce CAGE, a tool that is implemented as a Java application for reducing manual effort and inconsistent evaluations in grading programming assignments and exams. It provides a graphical user interface to the instructor for navigating through student code submissions that are stored in a folder. The submitted code is automatically split into multiple sections for fine-grained assessment and partial grading. Grades and feedback messages are inserted as code comments. Frequently used messages and the corresponding grades are listed and they can be inserted with a couple of mouse clicks. Assessment results and feedback can be exported as a file and directly sent to the respective students by e-mail. CAGE is used in the context of a course on data structures and algorithms by a professor and 6 teaching assistants for 2 semesters. The tool can be used after a 30 minutes long training session, and it can reduce the grading time by up to 50%. CAGE is open source and available online to facilitate its widespread use for fair and efficient assessment in introductory programming courses.

Keywords—*applied computing education, student assessment, CS1, computer programming, tool support*

I. INTRODUCTION

Introductory programming courses are not only fundamental to computer science but are also increasingly vital across various engineering disciplines. The rapid advancement of technology and the increasing reliance on software in various domains have led to an increasing importance of effective programming education [13]. As a result, the number of offered courses related to programming and the number of students enrolled in these courses increase. However, the assessment of programming assignments and exams in educational settings presents significant challenges. Traditional methods of manual code evaluation is time-consuming. Moreover, they are prone to subjective biases and inconsistencies, which can impact the fairness and accuracy of grading [19]. To address these issues, we introduce CAGE (Code Assessment and Grading Environment), a Java application that supports the instructor for evaluating and grading student code submissions.

CAGE can be used in the context of introductory programming courses to evaluate programming assignments or coding exam questions. The only assumption is that the submission or answer of each student must be placed in a text file and all such files regarding an exam question or assignment must be placed in a separate folder. CAGE provides a graphical user interface to navigate through the answers available in one folder at a time for streamlining the workflow [17]. The main window displays the code in an editable text area, accompanied by line numbers, which facilitates easy navigation and interaction [3].

Another panel of the CAGE main window is reserved for displaying feedback information, which is essential for effective learning. The tool incorporates a tree view display of as-

essment comments, allowing educators to manage feedback in a structured manner [18]. This hierarchical view facilitates the navigation and modification of comments, ensuring that feedback is comprehensive and easy to understand. The tool enhances visualization by color-coding line numbers based on the presence of comments, providing immediate visual feedback on the assessment status various code sections. Additionally, educators can insert grades and specific feedback at precise positions within the code, improving the clarity and precision of the grading process.

CAGE provides advanced functionalities for tool-assisted code grading. Educators can insert predefined comment phrases by double-clicking, which aids in the quick and consistent marking of common issues. The tool supports both read-only and editable modes, allowing educators to toggle between reviewing and modifying code. Additionally, an undo feature is provided to revert changes, ensuring that mistakes can be easily corrected. CAGE enables educators to export assessment results to CSV [21] files, which can be easily shared and analyzed. Furthermore, the tool includes an e-mail feature that allows for the sending of detailed feedback directly to the respective students.

An important component of CAGE is the splitter engine, which supports automated code splitting. This functionality covers parsing, analyzing, and splitting source code files to facilitate fine-grained assessment. The splitter engine employs exhaustive search algorithms to identify structural elements and potential issues within the code. For instance, it has mechanisms to handle syntax exceptions related to mismatched braces, ensuring the structural integrity of the code [8]. The splitter engine is responsible for dividing the code into manageable segments based on syntactic and semantic analysis. It uses a set of predefined rules and heuristics to determine the optimal points for splitting the code. Various strategies are employed to evaluate the quality and correctness of the code segments. These strategies are encapsulated in different modules, each implementing a specific algorithm for scoring and analysis. For example, one module might use a line-based approach to calculate the density of statements, while another might employ a more complex weighted calculation that considers the context and structure of code blocks [4]. Additionally, utility classes support these operations by managing key-value pairs and other fundamental data structures necessary for efficient processing.

CAGE is used in the context of a course on data structures and algorithms at Ozyegin University by a professor and 6 teaching assistants during the Fall 2023 and Spring 2024 semesters. It was used for evaluating 6 midterm examinations and 2 final examinations. All of these examinations were performed by students in front of computers, where they

write code using an integrated development environment. Examinations comprised a total of 45 coding questions. On average, each exam was attended by 68 students. We observed that CAGE can be used all the teaching assistants after a 30 minutes long training session. We also observe that it can reduce the grading time by up to 50% in comparison with the previous semesters where grading was performed on a text editor without any other tool support.

The remainder of this paper is organized as follows. We provide a summary of the related work in the following section. Then, we explain the code assessment method that is supported by CAGE in Section III. Section IV explains the features of CAGE. We explain the implementation and usage of the tool in Section V. In the same section, we also share our experiences in using CAGE in the context of a real course. Finally, we conclude the paper by discussing future work plans in Section VII.

II. RELATED WORK

The number of introductory-level programming courses and the number of students enrolled in these courses are increasing. This trend makes tool support essential to enhance the efficiency and consistency of assessment. There have been various tools introduced for this purpose and reported in the literature. These tools are designed to improve programming skills, monitor learning progress, and streamline the evaluation process for educators [8]. Automated programming assessment systems not only decrease the workload of instructors but also offer valuable insights into students' code quality from a learning perspective [4]. By automatically assessing the accuracy of students' programs, these tools play a crucial role in programming courses, particularly for novice learners [23], who can receive rapid feedback. There are also Web-based tools that can provide similar functionalities. These tools are employed in online coding events like competitions as formative assessment tools can be a valuable addition to traditional programming assignments in introductory courses [1]. These events present students with practical coding challenges and real-time feedback, fostering a more engaging and interactive learning experience. Furthermore, automated feedback generation is crucial in offering constructive feedback to students, helping them enhance their work and advance their programming skills [11]. Unfortunately, these tools mainly rely on dynamic analysis only [5]. They execute the submitted code with a set of test cases and the assessment is basically performed by observing the ratio of test cases that are completed successfully. However, these tools fall short in provisioning partial grades when the assessed source code cannot be compiled and executed. Although instructors can be more strict in grading programming assignments and utilize dynamic analysis/testing tools (i.e., submitted code that does not compile will simply result with a 0 grade), partial grading and detailed feedback is common and expected in exams of introductory programming courses. Students are supposed to receive partial grades even for incomplete code that follows a viable approach for tackling a sub-task required for solving a given problem.

Incorporating enhanced error messages into automated assessment tools can greatly benefit students in learning to program effectively [23]. These tools not only evaluate code but also provide informative feedback to assist students in understanding and correcting their mistakes, thereby enhanc-

ing their programming proficiency [19]. Additionally, tools that provide detailed feedback and interactive features, such as visualizations and problem-solving support, can further aid students in their learning process [12]. By providing clear explanations, step-by-step solutions, and pre-defined code segments, these tools can support students in grasping programming concepts more effectively. Additionally, tools that offer multi-perspective code analysis and interactive code visualization can assist developers in identifying and understanding code elements more efficiently [18]. Supporting the learning process by providing interactive feedback to students is out-of-scope of this work. CAGE focuses on the assessment of programming exams and assignments after they are submitted by students. In the next section, we explain the assessment method that we adopted.

```
public void push(int k, int data){
    /** ASSESSMENT
     * @grade 5 pts
     * @feedback
     */
    Node previous = null, tmp = top;
    for (int i = 0; i < k - 1; i++){
        previous = tmp;
        tmp = tmp.getNext();
    }
    /** ASSESSMENT
     * @grade 5 pts
     * @feedback
     */
    Node newNode = new Node(data);
    newNode.next = tmp;
    if (previous != null){
        previous.next = newNode;
    } else {
        top = newNode;
    }
}
```

Fig. 1. An example reference code that represents a possible correct answer for a coding question.

III. ASSESSMENT METHOD

In this section, we explain the assessment process that is supported by CAGE. As the preparation step, students submissions for each assignment or for each question of the exam must be saved in a text file. In the rest of the paper, we assume that an exam is being graded and we explain the process as such. Therefore, we assume that there is a folder for each exam question. The folder contains a set of text files, where each file includes the answer of a student for the corresponding question. Then, student responses are assessed on a question-by-question basis. In addition to the student responses, reference codes alongside answer keys are developed by the instructor. Each reference code represents a possible correct answer for the question. These reference codes are segmented into various sections, with each segment receiving a specific score. During this partitioning phase, it is crucial to divide the code into coherent segments. These code segments are generated by considering elements such as definitions, loops, conditions, inputs, and outputs. These segments are then assessed and assessment results are annotated as Javadoc [16] comments. Figure 1 shows an example reference code that represents a correct answer for a

coding question. It includes two segments with 2 points and 13 points.

```
public void push(int k, int data) {
    Node current = top;
    Node node = new Node(data);
    while (current != null) {
/** ASSESSMENT
 * @grade 1 pts
 * @feedback You lost by writing current =
    node the iteration
 */
        for (int i = 0; i <=k; i++) {
            current = current.next;
        }
        current=node;
/** ASSESSMENT
 * @grade 1 pts
 * @feedback Link is incorrect.
 */
        node.setNext(current.getNext());
    }
}
```

Fig. 2. An example code submitted by a student for a question where the expected answer is shown in Figure 1.

Subsequently, student responses undergo the same segmentation process facilitated by the tool, enabling a comparison at the segment level. The instructor assesses whether the student's code segments align with the reference segments. Hereby, multiple reference codes can be considered since multiple correct answers are possible for a programming question. Furthermore, the instructor provides feedback and assigns grades directly within the student's code as Javadoc comments. Figure 2 provides an example of a student code that has been segmented and graded as 0 according to the reference code in Figure 1.

CAGE streamlines this process through its functionalities, which include code browsing, segmentation, grading, commenting, compiling results into a report, and distributing results via e-mail. These are explained in the next section.

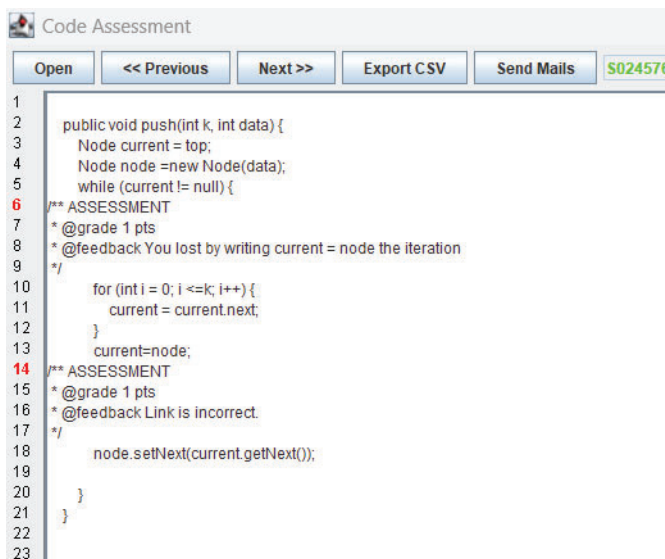


Fig. 3. A snapshot of the CAGE user interface.

```
1 split calculateBestSplit(code, refCode, evaluateSplit)
2     possibleSplits = {}
3     for line in code
4         if line not inside loop or conditional expression
5             possibleSplits += line
6         else
7             if line has a sibling
8                 possibleSplits += line
9     k = number of splits in refCode
10    possibleSplitSubSets = backtrack(possibleSplits, k)
11    minValue = ∞
11    for splitSubSet in possibleSplitSubSets
12        value = evaluateSplit(splitSubSet)
13        if value < minValue
14            minValue = value
15            bestSplit = splitSubSet
16    return bestSplit
```

Fig. 4. Calculating the best split of source code *code* according to the given evaluation function *evaluateSplit*, which calculates the value of a split on *code* with respect to reference code *refCode*

IV. CAGE FEATURES

In this section, we explain the main features of CAGE, which provides an intuitive graphical user interface. The main window displays code in an editable text area, accompanied by a line number area for easy navigation. Various buttons and options are used for managing the assessment workflow. A snapshot from the user interface of the tool is depicted in Figure 3. The user can open code files using the file chooser, save changes performed on the editor, and move to the previous/next file in the same folder. Assessment results can be exported to a CSV file. They can also be sent to all the students via e-mail. The line number area includes color-coded labels to provide visual feedback on the existence of comments. CAGE allows for the insertion of such comments that include grades and feedback at specific positions within the code. We explain the code splitting and code grading features in the following subsections.

A. Code Splitting

CAGE includes a component called the splitter engine, which enables automation for dissecting source code into distinct, manageable segments. It identifies logical blocks that can be independently analyzed or transformed. The identified code segments are viewed by the instructor during the assessment. This segmentation is essential for modular handling of code, allowing for more fine-grained analysis and operation [4].

Figure 4 shows the pseudo code of *calculateBestSplit*, which calculates the best split of a given source code according to a evaluation function, which calculates the value of a split of the given source code with respect to the reference code. First, we identify all possible split points as a list (Lines 2-8). In order for a line in the source code to be a possible split, certain conditions must be met.

- If the line is not inside of any conditional or loop expression, that line is directly taken as a possible split.
- If the line is inside of a loop or a conditional expression, in order for this line to be evaluated as a possible split, that line should have at least a sibling line, which is in

the same level of the original line (i.e. in the same loop, in the same condition etc). In this case, all sibling lines are considered as possible split points.

After all the split points are determined, backtracking is used to obtain the best split. The backtracking algorithm identifies all possible subsets of size k (number of splits in the reference code) of the possibleSplits list. As a last step, each possible subset is evaluated according to the given *evaluateSplit* function. The split with the minimum evaluation function value is returned.

In the current algorithm, there are 3 different types of *evaluateSplit* functions: *Dummy*, *Line*, and *Weighted*. *Dummy* returns a random value between 0 and 1.0 independent of the split subset. *Line* compares the number of lines in each segment of the split subset with the number of lines in the corresponding segment of the reference code and returns the sum of absolute differences between number of lines of the corresponding segments. *Weighted* is general variant of *Line*, where it differentiates between normal statements and conditional or loop statements by multiplying weights (α for normal statement, β for conditional or loop statements) while calculating the line differences between the segments of the split subset and reference code.

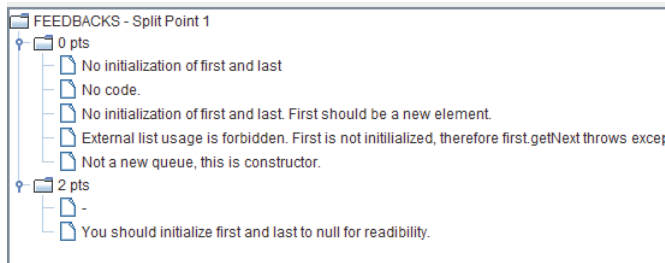


Fig. 5. Showing grades and comments on a segment basis on the treeview.

B. Code Grading

The tool offers helpful features for code grading. As illustrated in Figure 5, the automatically generated treeview is displayed. This treeview effectively integrates scores from prior reviews alongside related feedback information. Users can double-click to insert predefined comment phrases for assessment and grading, making the evaluation process quicker and more consistent. The tool automatically counts the number of comment phrases in the code, aiding in accurate grading. Users can choose between read-only and editable modes for different tasks, ensuring flexibility in the assessment process. The Ctrl+Z shortcut allows users to undo changes and revert the text area to its previous state, providing an easy way to correct mistakes.

V. IMPLEMENTATION DETAILS

CAGE is implemented in Java, leveraging its robust and platform-independent capabilities to create a powerful and flexible application. The main class handles the user interface and primary functionalities such as file operations, code grading, and custom rendering. Swing framework is used for developing the graphical user interface [6]. In addition, we also used java.awt libraries. These libraries provide foundational classes for creating user interface components and handling low-level events. Therefore, they complement the capabilities of the Swing framework by offering more

direct access to the native windowing system [7]. For file operations, java.io and java.nio offer classes for reading from and writing to files [15]. For e-mail functionality, javax.mail offer the necessary classes to send and receive emails, enabling the integration of e-mail communication within the application [9]. Finally, java.util.stream.Collectors from the Stream API allows for efficient processing and collection of data streams, supporting operations like filtering, mapping, and reduction [2]. We have implemented CASE with these core Java libraries. The Java source code of CAGE has been published at a public repository hosted by GitHub [10]. In the following, we share our experience in using the tool during the last two semesters.

VI. USING THE TOOL FOR THE ASSESSMENT OF REAL EXAMS OF A COMPUTER SCIENCE COURSE

CAGE has been used by a professor and 6 teaching assistants for 2 semesters. It only requires 30 minutes long training session for new users. Instructors use the tool in parallel, independently of each other. Each question is evaluated in turn. Thus, evaluation integrity on a question-by-question basis is ensured. The segmentation standard and scores provided with the reference code ensure consistency of evaluation. In addition, being able to easily navigate the codes with the tool and view the scores and feedback of the evaluations speeds up the assessment process. It can reduce the grading time by up to 50%.

CAGE was employed in the CS201 - Data Structures and Algorithms course during the Fall 2023 and Spring 2024 semesters. Throughout this period, students' performances in six midterm examinations and two final examinations were evaluated using this tool. The examinations comprised a total of 45 coding questions, which were further subdivided into 132 distinct segments. On average, each exam was attended by 68 students, totaling approximately 3,000 individual student code submissions. Each question was split into an average of four segments, and over 10,000 sample code segments were generated in Java and C++. All these exams, the corresponding questions and segments was evaluated by a professor, who instructs the course, and 6 teaching assistants. All the teaching assistants were able to use the tool without any follow-up questions or issues after a 30 minutes long training session. The professor, who conducted the grading without CAGE in the previous semesters for the same course, informed us that the grading time is reduced by up to 50%. The most helpful features for this speedup were listed as easy navigation among the answers, automated splitting of these answers, and the reuse of previously entered grades and comments for types of mistakes frequently repeated by students.

VII. CONCLUSION AND FUTURE WORK

We introduced CAGE, a tool that facilitates semi-automated assessment of programming assignments or exams. It reduces manual effort and inconsistent evaluations in grading student code submissions. CAGE provides a text editor that can navigate through submissions regarding a particular assignment or question. It includes features like automated code splitting, insertion of previously assigned grades and feedback for fine-grained and consistent partial grading. Additional features include the dissemination of

assessment results via exported files and automatically sent e-mail messages. CAGE is open source and available online to facilitate its widespread use for fair and efficient assessment in introductory programming courses. We already obtained promising results in terms of increased efficiency after using the tool for two semesters in the context of a second year course on data structures and algorithms. We are planning to use the tool in the following years as well. Our further future work plans regarding the tool include improving the current functionalities and increasing the level automation by automatically grading all the student submissions as described in the following.

Future developments will focus on incorporating sophisticated code splitting algorithms, graph-based models, and Machine Learning (ML) solutions. These enhancements will enable deeper analysis of code structures and logic, improve grading accuracy by suggesting grades to instructors, and provide more nuanced feedback to students. Our goal is to derive multiple models from the source code with static analysis. These models include abstract syntax trees (ASTs), control flow graphs (CFGs), and data dependency graphs (DDGs). We plan to use these models as input to ML models for guiding the instructor in the grading process.

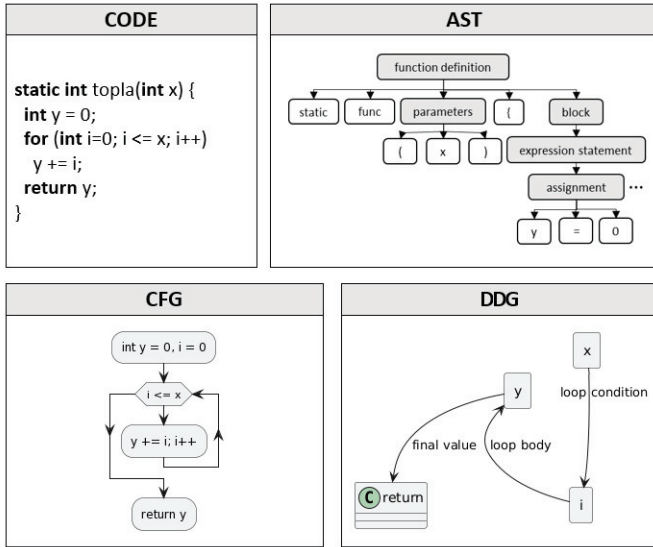


Fig. 6. Sample graph models for a code snippet (AST: Abstract Syntax Tree, CFG: Control Flow Graph, DDG: Data Dependency Graph)

Graph-based models will be a significant enhancement, enabling deeper analysis of the code's structure and logic. We plan to construct abstract syntax trees (ASTs), control flow graphs (CFGs), and data dependency graphs (DDGs) for each segment of code [22]. Figure 6 presents the relevant graphs for a sample code. These graphical representations allow for sophisticated analyses such as detecting code clones, identifying potential security vulnerabilities, and evaluating overall code quality [14]. ASTs will provide a hierarchical representation of the code's syntactic structure, CFGs will illustrate the control flow, and DDGs will capture the dependencies between different data elements.

The integration of ML techniques will be a cornerstone of our future development efforts. The process will start with compiling a comprehensive dataset from previous coding exams, including both correct and incorrect code submissions, each labeled with its corresponding score as determined by

expert evaluators. Using the ASTs, CFGs, and DDGs, we will extract relevant features that capture the structural and semantic properties of the code. These features will serve as input to our ML models. The trained ML models will then be integrated into CAGE to provide real-time scoring and feedback, enabling the tool to automatically grade code submissions with high accuracy and consistency.

To further enhance the grading process, we will implement algorithms to compute graph model similarities. This will involve calculating the graph edit distance (GED) between the student's code and the reference solution. By quantifying the structural differences between these graphs, we can provide more nuanced feedback and assign partial scores based on the degree of similarity [20]. Additionally, we will explore the use of graph kernels to measure similarity and incorporate these metrics into our ML models to improve their predictive performance [22].

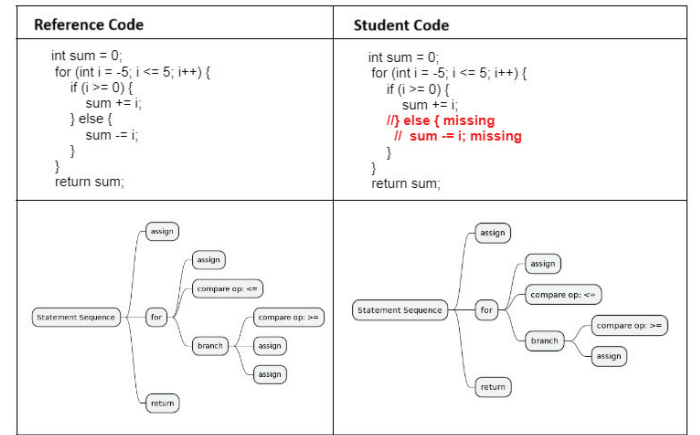


Fig. 7. A sample comparison between two AST models.

Figure 7 presents the AST graphs of a student's response in comparison with a reference code. The discrepancies in the student's response are visually identifiable in the graph. Likewise, CFG and DDG can be utilized to analyze these differences. By employing methods such as GED to measure the deviations between these graphs, a quantifiable value can be obtained, which can be effectively used for grading purposes.

We also plan to develop a Web-based version of the tool. By pursuing these enhancements, we aim at turning CAGE into a comprehensive, user-friendly, widely available and accessible application that supports multiple programming languages. This will significantly reduce the manual effort involved in code evaluation, ensure fair and consistent grading, and ultimately enhance the quality of programming education. The continued development and refinement of this tool have the potential to make a substantial impact on educational practices and assessment efficiency in computer science and related fields.

ACKNOWLEDGEMENTS

We thank the teaching assistants of the CS201 (Data Structures and Algorithms) course offered at Ozyegin University over the last 2 semesters for supporting us in evaluation of CAGE by participating in grading tasks of some exam questions with the help of this tool.

REFERENCES

- [1] D. Anitha and D. Kavitha. Online coding event as a formative assessment tool in introductory programming and algorithmic courses—an exploration study. *Computers Applications in Engineering*, page 1010, 2020. Access with Article Galaxy.
- [2] J. Bloch. *Effective Java*. Addison-Wesley, 2018.
- [3] M. Blumenstein, S. Green, H. Nguyen, and H. Nguyen. An analysis of the effectiveness of automated formative feedback for students. In *Proceedings of the Sixth Australasian Computing Education Conference (ACE2004)*, pages 25–33, 2004.
- [4] H. Chen, B. Nguyen, Y. Yan, et al. Analysis of learning behavior in an automated programming assessment environment: A code quality perspective. *IEEE Access*, page 212170, 2020. Open Access.
- [5] S. Combéfis. Automated code assessment for education: review, classification and perspectives on techniques and tools. *Software*, 1(1):3–30, 2022.
- [6] R. Eckstein, M. Loy, and D. Wood. *Java Swing*. O'Reilly Media, 1998.
- [7] D. Geary. *Graphic Java 2, Volume 2: Swing*. Prentice Hall PTR, 1999.
- [8] S. Gupta and A. Gupta. E-assessment tools for programming languages: A review. *Annals of Computer Science and Information Systems*, 14:80130, 2018. Open Access.
- [9] J. Jason and J. Jason. *JavaMail API: Sending and Receiving Email with Java*. O'Reilly Media, 2003.
- [10] Umit Kanoglu. Codeassessment. <https://github.com/knglumt/CodeAssessment>, 2024. Accessed: 2024-07-16.
- [11] H. Keuning, J. Jeuring, and B. Heeren. Towards a systematic review of automated feedback generation for programming exercises. In *Proceedings of the 2016 ACM Conference on Innovation and Technology in Computer Science Education*, page 41–46, 2016.
- [12] H. Keuning, J. Jeuring, and B. Heeren. A systematic literature review of automated feedback generation for programming exercises. *ACM Transactions on Computing Education*, page 15621220, 2018. Open Access.
- [13] J. W. Kim, A. Berglund, A. Pears, and T. Efraimsson. The effectiveness of automated feedback in introductory programming courses: a systematic review. In *Proceedings of the 2016 ACM Conference on Innovation and Technology in Computer Science Education*, pages 106–111, 2016.
- [14] S. Kulal, A. Deshpande, M. Vasic, S. Chaudhuri, and B. Ray. Spoc: Search-based pseudocode to code. In *Advances in Neural Information Processing Systems*, pages 9992–10002, 2019.
- [15] P. Niemeyer and J. Knudsen. *Learning Java*. O'Reilly Media, 2005.
- [16] Oracle. *Javadoc - The Java API Documentation Generator*, 2010. Rev. 1.5.0.
- [17] R. Pettit, J.D. Homer, K.M. Holcomb, N. Simone, and S. Mengel. Are automated assessment tools helpful in programming courses? *ASEE Annual Conference and Exposition, Conference Proceedings*, 122, 01 2015.
- [18] A. Qayum, S. R. Khan, I. Rehman, et al. Finecodeanalyzer: Multi-perspective source code analysis support for software developer through fine-granular level interactive code visualization. *IEEE Access*, 2022.
- [19] Y. Qian and J. D. Lehman. An investigation of high school students' errors in introductory programming: A data-driven approach. *Journal of Educational Computing Research*, page 6050, 2019. Access with Article Galaxy.
- [20] B. Ray, D. Posnett, V. Filkov, and P. Devanbu. On the "naturalness" of buggy code. *Communications of the ACM*, 59(9):31–37, 2016.
- [21] Y. Shafranovich. Common format and MIME type for comma-separated values (CSV) files. RFC 4180, RFC Editor, October 2005.
- [22] Y. Wan, Z. Zhu, P. Yin, Y. Chang, X. Wang, and X. Li. Multi-modal graph-based reasoning for code summarization. *arXiv preprint arXiv:2104.11351*, 2021.
- [23] Z. Zhou, S. Wang, and Y. Qian. Learning from errors: Exploring the effectiveness of enhanced error messages in learning to program. *Frontiers in Psychology*, page 5030, 2021. Open Access.